

Chapter 7

CS1 course package - Casper College

Charlotte Gruner
Casper College, Casper WY

Abstract

This chapter presents the integration of Parallel and Distributed Computing (PDC) concepts into an introductory computer science course (CS1) at Casper College, a two-year public institution. The primary goal was to embed foundational PDC topics into the existing curriculum without displacing traditional core content such as loops, arrays, functions, pointers, and classes. The pedagogical approach utilized a combination of unplugged activities, a custom digital textbook module, and plugged-in programming assignments in both C++ and Python. This chapter details two distinct implementations: one across an 8 week summer session and one spanning a full 16 week semester. Preliminary analysis of survey data collected in a baseline semester in which no PDC interventions were implemented and a semester in which several of the described interventions were implemented indicate that student perception of learning gains in non-PDC related topics remained generally the same while student perception of learning gains in PDC topics was higher, in general, in the semester in which PDC interventions were implemented.

Descriptor Elements

Programming Language Employed: predominantly C++, some Python (for Physical Computing projects, Spring 2026)

Textbook used: zyBooks[2] (Summer 2025) and REVEL edition of Starting out with C++ by Tony Gaddis[1] (Spring 2026)

Relevant core courses: CS1 (COSC 1030)

Pre-requisites: CS0 (COSC 1010) - this course is taught using Python/Scratch

Relevant PDC topics:

Relevant PDC topics and Blooms levels Table 7.1 lists the integrated PDC topics with bloom's classification (Remember (RE), Understand (UN), Apply (AP), Analyze (AN), Evaluate (EV), and Create (CR)).

Learning outcomes:

1. Define and recognize fundamental PDC terms such as multi-core, data parallelization, and task parallelization.
2. Apply OpenMP directives to parallelize array processing and analyze the resulting performance gains through benchmarking.
3. Identify and explain race conditions and shared resource conditions in both physical simulations and C++ code.
4. Implement and manage multiple processes in Python using the multiprocessing package for event-driven physical computing tasks.

Table 7.1: CASPER CS1 Relevant PDC Topics and Bloom's Levels

PDC Concept	Chapter Section				
	7.3.1 zyBooks, 7.3.2 Un- plugged Penny	7.3.3 Un- plugged Coin Add.	7.4.1 Plugged- In Array Creation, 7.4.2 Plugged- In Array Sum	7.4.3 Plugged- In Earth- quake	7.4.5 Phys Comp 2, 7.4.6 Phys Comp 3
PDC Vocabulary/Foundations	RE, UN			RE, UN	
Data Parallelism	RE		AP, AN		
Race Conditions/Synchronization		UN	UN, AN		
Speedup and Benchmarking				AP, AN	
Distributed Computing				UN, AP	
Event-Driven Programming					UN, AP
Task Parallelism	UN				AP, AN

Context for use: Casper College is a 2-year public, community college located in central Wyoming. The total enrollment at Casper College is typically between 4,500 students and 5,000 students. The most popular programs are in general studies, health care, and agriculture. The Computer Science Department, which is housed within the Mathematics Department in the School of Science generally consists of around 50 declared majors with 80% working on freshman level coursework and 20% working on sophomore level courses. Demographically, Casper College has more students who are older than the typical 18–21 age group found at a typical university in the United States and less than half of the students are full time. Casper College also has a significant number of high school students enrolled in classes taught at their institutions (dual enrollment) and who attend classes taught at Casper College (concurrent enrollment). The CS1 course is taught at one of the local high schools by a high school instructor and on the Casper College campus by a college instructor. The Computer Science Department has one full time faculty member. The CS1 course is a requirement for the A.S. degree in computer science and also a requirement for the A.S. degree in engineering. These are the two main populations of students who take CS1 at Casper College. The Computer Science Associates Degree at Casper College is meant to serve as the first two years of a 4-year degree program and includes the following core computer science classes: CS0 (COSC 1010) Introduction to Computer Science, CS1 (COSC 1030) Computer Science I, CS2 (COSC 2030) Computer Science II, (COSC 2150) Computer Organization, (COSC 2300) Discrete Structures. Class sizes for 1000-level courses tend to be below 20. Class sizes for 2000-level courses tend to be under 10. At Casper College the CS1 course, CS1 (COSC 1030), is considered a 3 hour lecture, 2 hour lab class that

transfers as a 4 credit class. This class is generally offered in two formats during the spring semester: an on-line, asynchronous format and in a live class format that meets every weekday for 50 minutes. During the summer semester, the class is offered in an 8-week online, asynchronous format.

7.1 Introduction

While integrating Parallel Distributed Computing (PDC) early in the computer science curriculum is vital, doing so in the first two years without displacing traditional core topics remains a challenge because the curriculum in these classes is already packed and because there is a lack of curricular materials for this level that are easily incorporated into these classes. To address this, CDER initiated a two-year program to develop, test, and broadly disseminate adaptable curricular elements. Faculty teaching CS1 and CS2 at diverse institutions were recruited as “Testing Teams,” supported by “Development Teams” who created and tested the classroom materials. Casper College participated in this effort as a Testing Team. This chapter explains Casper College’s integration of modules adapted from the Development Team as well as new PDC materials into a traditional CS1 course. The overall goal of this effort was to identify opportunities to add PDC content without substantially changing the existing curriculum. The PDC interventions were implemented in an 8-week summer 2025 session (with 4 students at the beginning of the course, 2 finishing) and a spring 2026 semester course (11 students at the beginning, 5 who finished). The spring 2025 course (19 students at the beginning of the course, 13 finishing) served as the baseline and had no PDC interventions.

7.2 How CS1 was changed and PDC topics integrated

In the summer 2025 session the textbook zyBooks[2] *Programming in C++* was the primary textbook. The PDC interventions that were added to this course consisted of:

- a new instructor-written module for the zyBooks text (zyBooks Section 1.9 Parallel Computing, Section 7.3.1),
- a distributed computing assignment involving earthquake data and the USGS API (Distributed Data Earthquake Assignment Section 7.4.3).

In the 2025 and 2026 spring semesters, the primary textbook was the REVEL edition of *Starting out with C++* by Tony Gaddis[1]. For this course the main PDC interventions added in the spring semester 2026 were:

- Unplugged Coin Search Activity (see Section 7.3.2),
- Plugged-in OpenMP Array Creation Assignment (see Section 7.4.1),
- Unplugged Coin Addition Activity (see Section 7.3.3),
- Plugged-in OpenMP Array Sum Assignment (see Section 7.4.2).

The other material that was changed in this class was the addition of three assignments in Physical Computing to expose students to event handling. These were done in Python using Raspberry Pi with GPIO controlled hardware (Section 7.4.4, Section 7.4.5, Section 7.4.6).

7.2.1 Integrated Syllabi (Pre and Post)

A summary of the summer 2025 PDC material added and the timing of the material relative to the baseline CS1 material is shown in Table 7.2.

Table 7.2: CASPER CS1 Baseline versus Summer 2025 PDC Intervention

Week	Baseline material	Post (PDC content added Summer 2025)
1	Introduction to C++, variables and assignment (zyBooks Ch 1,2)	zyBooks Section 1.9 Parallel Computing (7.3.1), Unplugged Penny Search Activity(7.3.2)
2	Branches (zyBooks Ch 3)	—
3	Loops (zyBooks Ch 4)	—
4	Arrays/Vectors (zyBooks Ch 5)	—
5	Functions (zyBooks Ch 6)	Distributed Data Earthquake Assignment (7.4.3)
6	Object/Classes (zyBooks Ch 7)	—
7	Pointers (zyBooks Ch 8)	—
8	Final exam week	—

A comparison of the baseline course from spring 2025 to the course with interventions in spring 2026 is shown in Table 7.3.

Table 7.3: CASPER CS1 Baseline versus Spring 2026 PDC Intervention

Week	Baseline (Spring 2025)	PDC Interventions (Spring 2026)
1	Intro to computers and programming/Environment set up (Gaddis Ch 1)	—
2	Intro to C++/on going project work (Gaddis Ch 2)	Physical Computing #1 (7.4.4)
3	Expressions and Interactivity (Gaddis Ch 3)	Physical Computing #2 (7.4.5)
4	Decision Structures (Gaddis Ch 4)	—
5	Loops and Files (Gaddis Ch 5)	—
6	Function (Gaddis Ch 6)	—
7	Arrays and Vectors (Gaddis Ch 7)	—
8	Arrays and Vectors (Gaddis Ch 7)	—
9	Pointers (Gaddis Ch 9)	Physical Computing #3 (7.4.6)
10	Pointers (Gaddis Ch 9)	Unplugged Penny Search Activity (7.3.2), Plugged-In OpenMP Array Creation Assignment (7.4.1)

11	Structs (Gaddis Ch 11)	Plugged-In OpenMP Array Sum Assignment (7.4.2)
12	Structs (Gaddis Ch 11)	
13	Classes (Gaddis Ch 13)	Unplugged Coin Addition Activity (7.3.3)/Plugged-In OpenMP Array Sum Assignment (7.4.2)
14	Classes (Gaddis Ch 13)	—
15	Searching/Sorting(Gaddis Ch 8)	—

7.2.2 Highlights

The same core CS1 material was covered in all the CS1 courses. In the no-PDC-intervention of Spring 2024, the project work was a student-directed group project involving a Raspberry Pi GPIO controlled hardware. In the PDC-intervention group this project was replaced with directed Physical Computing assignments. In the summer 2025 course, the hardware-based project work was dropped because of the shortened schedule, students were on-line only, and the zyBooks textbook was a new resource for the instructor that semester.

7.2.3 Challenges

The unplugged activities generally worked well, required some class time but rarely the entire 50 minute class, and student engagement was high. A challenge for unplugged activities is adaption for on-line students. The OpenMP-based plugged-in assignments have a steep learning curve and require a lot of explanation. The physical computing activities also have a steep learning curve since they require some knowledge of circuits. These assignments also require students to go back to programming in Python (the primary language of the CS0 course at Casper College), an additional cognitive load, but also an opportunity to help students see how different languages are better suited for different tasks. An additional challenge in adopting PDC content is the time required for an instructor to understand the materials developed elsewhere and to adapt them to the particular class or to create new materials on this content.

7.3 New Unplugged Activities

7.3.1 zyBooks Section 1.9 Parallel Computing

Problem Description, Prerequisites, and Learning Outcomes

Problem Description zyBooks has the capability for instructors to add sections to the text. In the summer semester of 2025, the primary text book was zyBooks, Programming in C++ [2]. In the first introductory chapter of this text, section 1.8 is titled “Computer Tour” which has content related to computer hardware. Since this section discusses the concept of a “processor”, we modified TNTECH’s introductory slides on PDC used in conjunction with the Unplugged Penny Search Activity to create a section 1.9 called “Parallel Computing”.

Prerequisites. Knowledge of basic computer hardware, parts of a computer.

Learning Outcomes.

- Students will be introduced to PDC vocabulary including “sequential computing”, “parallel computing”, “core”, “multiprocessing”, “data parallelism”, “synchronization”, “load imbalance”, and “task parallelism.”

Implementation Details

- **Logistics (Materials required for the activity, etc.)** This module can be shared by the author with any interested instructors by emailing the instructor at charlotte.gruner@caspercollege.edu. The estimated time for students to complete the section is under 30 minutes.
- **How to Conduct** Students were assigned to read the section along with the rest of the chapter reading and complete the formative assessments in the form of participation activities within the chapter.
- **Instructional material (Lectures, Videos, Assignments, etc.)** Shared zyBooks module or new custom creations.
https://github.com/charlottegruner/CDER_CC_CS1/tree/main/ZybooksChapter
Figure ?? and Figure ?? show the custom zyBooks material to introduce topics in Parallel Computing.

Experience and Teaching Tips

Adding a section that included text, images and formative assessments in in the form of participation assignments to the zyBooks textbook by using previously created graphics and text was straightforward.

Data and Analysis

No formal data was collected on this specific activity. This was a small on-line class with only two students. Students were not aware that this section was not part of the normal materials in the zyBooks text and read and completed the assessments seamlessly along with the rest of the chapter materials. This intervention would be easily scalable to a larger class size.

7.3.2 Unplugged Penny Search Activity

Problem Description, Prerequisites, and Learning Outcomes

Problem Description This activity is described in detail in section 1.3.2. We adapted TNTECH’s materials described in section 2.3 for use in class and as an guided at home activity for on-line students. Briefly, a group of students are given the task of finding a specific penny from a group of pennies (such as the oldest). Students are guided toward

1.9 Parallel Computing

Goal: Define PDC vocabulary including processor, CPU, core, sequential computing, parallel computing, give some examples of parallel and distributed computing.

The section titled "Computer Tour" introduced the basic hardware components in a computer system, including the processor, where the actual computations are performed. The processor is also referred to as the **Central Processing Unit** or **CPU**. The main electronic component inside the CPU is the transistor. In general, the more transistors that can be put into a chip the more powerful the computer. However, more power requires more energy and produces more heat that must be dissipated. One of the solutions for increasing performance is to put multiple processors into the CPU. When a CPU contains multiple processors, they are often called "**cores**" in the specifications for the chip, as in "the Intel Core i7 processor has 8 cores." However, "core" and "processor" are also often used interchangeably, so this type of computer system may also be termed a **multiprocessor**. Having more than one core allows programs to be divided among these individual processors. This arrangement allows tasks to run at the same time on different cores and is one form of **parallel computing**. **Parallel computing** is a computational model where tasks are broken up into smaller pieces that can each be run on a different processor at the same time (in parallel). These tasks might be entire programs or they might be a program that has been broken down into smaller pieces.

Sequential Computing	Parallel Computing
Sequential computing is the standard programming model where the CPU executes each operation of the program in order, one at a time on the same processor.	Parallel Computing is a computational model that breaks programs into smaller sequential operations and performs those operations at the same time, in parallel on different processors.

Often in computing the same computation needs to be done over and over. **Data parallelism** is when a dataset can be divided and a computational task performed on each section separately by different processors. Consider a program that will add together 1000 numbers. If there are 10 processors and they each add 100 numbers to find the sum at the same time this operation will take much less time than one processor adding all 1000 numbers. Of course the 10 individual sums also must be added together. This part of the process is called the **synchronization** phase. Consider this same example if we had 1000 processors. It would not be advantageous to split the task between 1000 processors because the synchronization phase would likely take at least as long as having one processor do the task sequentially. In addition, if there are 10 processors, but the task is divided unevenly, for example if processor 1 is given 500 of the numbers and the rest of the numbers are distributed among the remaining 9, the processors, the entire task will be waiting for processor 1 to finish. This is an example of **load imbalance**, when one process

A program is generally composed of many individual tasks. Sometimes it is possible for these tasks to be performed at the same time. In this case, it is possible to implement a parallel processing solution known as **task parallelism**, where each processor can handle individual tasks. Note that this depends on whether or not the program can be broken up into different tasks. A load imbalance can also occur in this case if one of the processors has more processing to do than other processors.

Further reading:

Adams J.C., Brown R., Matthews S.J., Shoop E., and others. *PDC for Beginners*, 0.1 Edition. Available online: <https://dx.doi.org/10.55682/VXWY1300>

Figure 7.1: First part of new custom zyBooks section 1.9 Parallel Computing

increasing the speed at which this task can be done by employing more students in the group to perform the task to develop some intuition about parallelization and provide an opportunity to introduce students to some vocabulary and concepts related to PDC.

Prerequisites No prerequisites are required for this activity.

Learning Outcomes

Implementation Details

- **Logistics (Material required for the activity, etc.)** For the live class approximately 50 pennies per group is required.

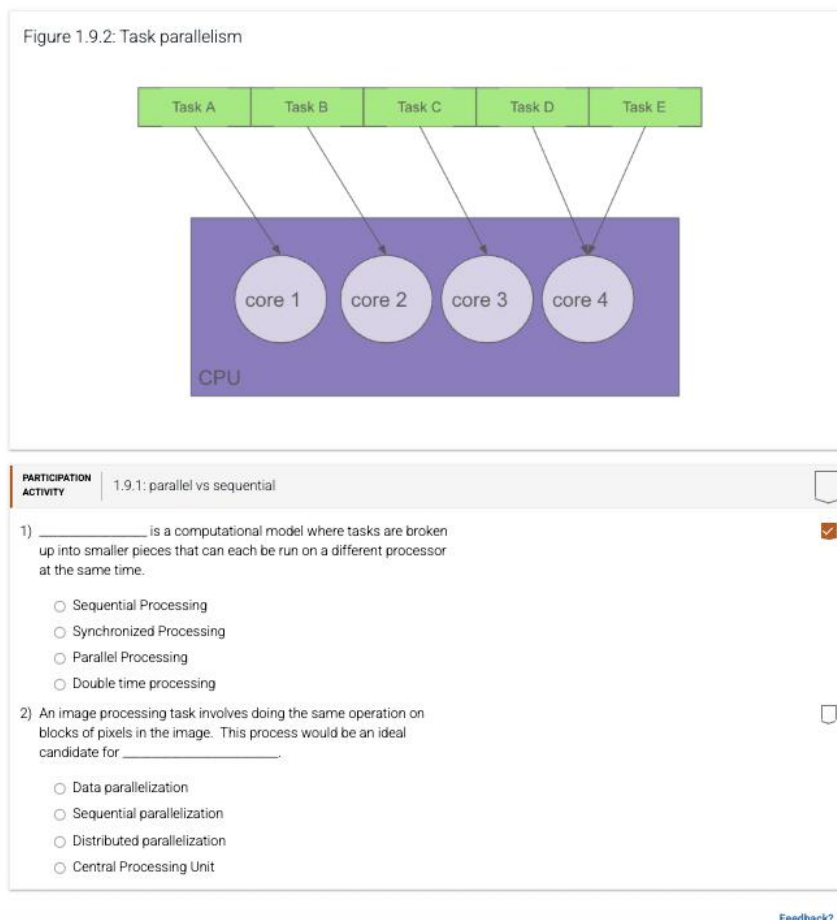


Figure 7.2: Second part of new custom zyBooks section 1.9 Parallel Computing

- **How to Conduct** Students are put into groups of 4-5 initially. In several steps, students are guided from constructing a sequential search where one student is timed in the task of finding the oldest penny to performing the task with multiple students participating, timing each experiment. Students should discover that adding more students to the task works for a while, but when taken to the extreme (each student having one penny) the task takes longer.
- **Instructional Material (Lectures, Videos, Assignments, etc.)** The adapted materials for on-line students are at: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper/Unplugged_Activities/Unplugged_Penny_Search

Experience and Teaching Tips

We have found this unplugged activity is effective with groups ranging from high school students with no prior knowledge or experience to CS2 students. With CS1 students we have used this activity successfully at two different times in the semester. In the first week of class, this activity fits with the introductory material on computer hardware (as it

was used here in the summer semester 2025). When the hardware concept of processor is discussed, it is natural to extend the information that most modern computing systems have multiple processors. In Spring 2026 semester, we used this activity as the lead in to the first experience with OpenMP and parallelizing array initialization after arrays were introduced approximately midway through the class.

Data and Analysis

No specific formal data was collected on this activity. Students were observed to have a high level of engagement in the activity which naturally brings up a variety of PDC topics and was observed to be meeting learning outcomes.

7.3.3 Unplugged Coin Addition Activity

Problem Description, Prerequisites, and Learning Outcomes

Problem Description. This unplugged activity simulates the addition of all elements in an array. The purpose of this activity is for students to recognize a situation in which there is a race condition because of a shared resource (the accumulator storage location in this example).

Prerequisites. It is helpful but not necessary if students have done the Unplugged Coin Search Activity. It is also helpful but not necessary that students have been exposed to array syntax to sum the contents of an array using an accumulation loop. It may additionally be motivating for students to for students to have tried to naively parallelize this activity as shown below using OpenMP and discovered that it doesn't work.

```
// THIS CAUSES A RACE CONDITION
#pragma omp parallel for
for (int i = 0; i < array_size; i++) {
    total += array[i]; // 'total' is shared and unprotected!
}
```

Learning Outcomes.

- **Identify** situations where parallel threads encounter a race condition due to a shared resource.
- **Explain** the “read-modify-write” flaw that occurs when multiple processes access an unprotected accumulator.
- **Describe** why simple loop parallelization (like naive OpenMP directives) fails without proper synchronization.

Implementation Details

- **Logistics (Material required for the activity, etc.)** Enough containers (paper cups) with approximately 10 coins of mixed denominations (pennies, nickels, dimes,

quarters) for each student participating. Ideally, there will be enough coins for each student to prevent a student from being able to look at the coins and immediately know how much they sum to, but not enough so that they will depart for the vending machine.

- **How to Conduct** Students should be in groups of 4-5. Each group should be given a cup with coins and 4 empty cups to use to distribute coins to team members. The problem is how to distribute the work of figuring out how much money is in the group. Students will recognize that dividing the coins among group members and having each student count their cup should decrease the time needed to figure out the cumulative sum. However, students should be reminded that the loop to add the contents of an array, looks like:

```
total = 0
for each value in the array
    total = total + value
```

The important line to point out is that before the current element of the array can be added to *total*, the process needs to have access to the current value of *total*. To simulate this, have one student (or the instructor if the class is small) act as the keeper memory location for *total*, in charge of giving the total to each process (student) and recording the new value as each student accesses it. Each process needs to request the current contents of *total*, add the current value to it and then store it back in *total*.

Students should hopefully observe that *total* is not being accumulated correctly as each student counter competes to get the current contents of *total* and then store their updated value.

- **Instructional material (Lectures, Videos, Assignments, etc.)** A video of the activity may be helpful for asynchronous online students, but synchronous on-line students may be able to participate if warned ahead of time to have coins available to use. Materials for this activity are found here.

Experience and Teaching Tips

So far, the instructor has tried this with a CS1 class (Spring 2026) and a CS2 class (Fall 2025). Both times, the most difficult part was getting students to understand the directions surrounding the *total* accumulator variable. Both class sizes were small, so the instructor acted as the storage location for this variable. Students emailed the instructor to request the value of the accumulator, the instructor sent back the value, the student did the addition and sent back the new value to store. The concept was to create an email trace that the class could follow after the experiment. Unfortunately, students were confused and only a couple of them understood what to do. Future attempts might include changing the instructions to include values written on a paper instead of coins, or a more complicated arithmetic that would slow down the students' tasks. Future attempts might also have the total stored on a small whiteboard that could be requested from the keeper of the accumulator. Even though

this is work in progress, in both classes that this activity has been tried, students ultimately seemed to grasp the problem and were able to also realize that the simple solution of “locking” the accumulator so only one process could use it would not result in any performance gains, so they realize the solution would be for each process to have a separate subtotal and add those together after each process was done with it’s part of the array.

Data and Analysis

No formal data was collected on this activity. In both classes students were engaged in the simulation, especially the second time after the instructions were clarified. In both classes the students were ultimately able to figure out the issue with the shared resource and suggest a the solution of having processors computing partial sums.

7.4 New Plugged-In Activities

7.4.1 Plugged-In OpenMP Array Creation Assignment

Problem Description, Prerequisites, and Learning Outcomes

Problem Description In this activity students are asked to write a C++ program that allocates space for a large array and then fills the array with the same value as the array index. The students are shown how to use the `#pragma omp parallel for` to parallelize the loop and are instructed to figure out how many different threads are created and which threads are handling which indices. The students are introduced to the use of the `chrono` library for timing code, so that the speedup can be documented. These materials were adapted from TNTECH’s Plugged-In OpenMP Data Parallelism Assignment in Section 2.4.2.

Prerequisites Experience with C++ arrays and dynamic memory allocation using pointers.

Learning Outcomes

- Students will define and explain parallel computing terms: parallel processing, sequential processing.
- Students will identify the number of cores on their particular machine.
- Students will learn to use the OpenMP `#pragma omp parallel for` to do data parallelization
- Students will learn how to compile using the `-fopenmp`.
- Students will observe how the loop is split into chunks that are handled by different threads.
- Students will use the C++ `chrono` library to measure and compare the execution time of sequential versus parallelized code.

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** Materials are found at: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper/Plugged_Activities/Plugged-In_OpenMP_Array_Creation In these materials students are encouraged to work in pairs to promote discussion as they are work on this assignment. Part 1 of the assignment shown in Figure 17.3 asks the students to do some preliminary research on PDC-related vocabulary and also some vocabulary specific to OpenMP to acquaint them with important terms. Students are allowed to use AI to prompt for definitions.

Part 1: Learn about the OpenMP API

OpenMP is an API that supports shared-memory parallel processing for C/C++ and Fortran. .

1. Define API [AI]
2. Define parallel processing [AI]
3. Define shared-memory [AI]

OpenMP allows programmers to parallelize code by adding pragmas to existing code.

4. What's a `pragma`? [AI]
5. What is the goal of parallelization of code? [AI]
6. What's a "core"? [AI]
7. How many cores does the computer you are using have? Some ideas might be to find out what kind of processor your computer has (usually buried in a 'settings' tab). Or if you are on a linux machine you can explore the commands `nproc`, `lscpu`, and `htop` or `top`. [AI]

The programming we have done so far in COSC 1010 and COSC 1030 has been sequential.

8. What does "sequential processing" mean? [AI]

Figure 7.3: CASPER CS1 Part 1 of Plugged-In OpenMP Array Creation Assignment

Part 2 of the assignment (shown in Figures 17.4 and 17.5), students are instructed to write a sequential program to construct and initialize a large array. The goal of this task is for students to write a program that takes a noticeable amount of time to complete. Prior to this activity, students may not have written any programs that did not appear to run almost instantly. The next steps walk the students through thinking about whether or not this task could be done in parallel and how to implement this using the appropriate `#pragma` from OpenMP. In this step, the students are guided to use a smaller array so that they can document via print statements which threads are performing which array index initializations, which should ideally be the same as the number of cores that their computer has (which students should have determined in Part 1).

Part 3 of this assignment is for the students to learn to use the `chrono` library to benchmark the parallel and sequential code (as shown in Figure 17.6). They should find that there is speedup in the parallel version, but not as much as they predicted. In this step, students are encouraged to spend some time using AI tools to learn about the `chrono` library to figure out how to use it for this application.

- **How-To Guides**

Part 2: Write a short program that will hopefully take a while if run sequentially. Then explore a few aspects of OpenMP to parallelize a section of the code.

In parallel computing, the goal is to split up the task into subtasks that can be performed by different processors. OpenMP is an API that allows a programmer to use compiler directives to specify parts of the code that should run in parallel.

9. Write a program that allocates memory for 100 million integers and a loop that assigns each index in the loop to contain the integer corresponding to its index. So at index 0 the value in the array is 0, at index 1, the array is 1 and so forth. Note that you might have to explore the `long` and `long long` data types since the numbers we are dealing with here can be quite long. Feel free to use AI tools for this task to find out what the largest integer that can be stored on a given system. Remember that these values are not standard for C++, so you will need to figure out what those maximum values are on the particular system you are using. Also, remember that your system may not be able to allocate this much stack space, which will require using heap memory through dynamic memory allocation (using pointers). I encourage you to experiment with trying the program with stack memory to see what happens.

10. Once you have written the program in 10, consider the loop that fills the array. Is this an example of sequential processing or parallel processing?

11. Could this program be parallelized? Could tasks in this program be done by different cores at the same time? (Think back to the penny searching experience.)

12. The compiler directive to parallelize a loop is:

```
#pragma omp parallel for
```

The directive is placed directly above the `for` line of the loop that fills the array. Note that this pragma works for `for` loops and not while loops, so if you wrote the loop as a `while` loop, re-write it as a `for` loop. You will also need to add an include:

```
#include <omp.h>
```

To compile the program, you will need to add a new compiler option `-fopenmp`. If the source code is located in the file `main.cpp`, the command to compile will be:

```
g++ -fopenmp main.cpp -o main
```

Note that using OpenMP on a Mac requires some setup because the default clang compiler on MacOS does not support OpenMP by default. Follow [these directions to set up a Mac](#).

Figure 7.4: CASPER CS1 Part 2 of Plugged-In OpenMP Array Creation Assignment

Experience and Teaching Tips

This exercise would work well as a 2 hour in-class lab. Another option would be for students to complete Part 1 of the activity before class so they would be prepared to start the coding part in class.

Data and Analysis

The Spring 2026 PDC Intervention CS1 course had only seven students. Three of the four in-person students completed the assignment successfully, reporting the time taken to do the assignment as 1.5 - 3 hours. We did give students in-class time to work on the assignment so that we could monitor and guide their progress. One of the three on-line student completed the assignment, reporting that it took 2.5 hours. The on-line student also requested that the instructions be clearer.

7.4.2 Plugged-In OpenMP Array Sum Assignment

Problem Description, Prerequisites, and Learning Outcomes

Problem Description Students are instructed to write a C++ program to add all of the elements of the array and then to parallelize that loop using OpenMP, as learned in the Plugged-In OpenMP Array Creation Assignment shown in Section 7.4.1. They will make the discovery that this does not work as desired and the accumulator (which is being shared by all the threads) is a shared resource that is experiencing a race condition. Then, students participate in the Unplugged Coin Addition Activity shown in Section 7.3.3.

13. There is another way to find out how many processors your computer has. You will need to include the `<thread>` library.

```
#include <thread>
```

The value `thread::hardware_concurrency()` stores the number of processors. Add a line of code to print this out and see if it agrees with the answer you found above.

14. The `#pragma omp parallel for` basically splits the iterations of the loop so that they will be done in parallel by different threads. A thread is like a mini-program. To get to see how this works first reduce the size of the array (and the number of iterations of the loop) to 50. We are going to add a print statement to the loop and we don't need to watch millions of iterations. Inside the loop add the line of code:

```
id = omp_get_thread_num();
```

The function `omp_get_thread_num()` returns the thread number that is executing an iteration of the loop.

Print out the `id` inside the loop. How many different numbers are printed out? Does this make sense given the number of processors?

15. Add some additional text to your `cout` statement such as

```
cout << "id: " << id;
```

Does the output seem all jumbled up? Why do you think that could be happening?

To fix this problem, we can change to using a different function called `printf()`, that is actually part of the C programming language. The equivalent of the above `cout` using `printf` is:

```
printf("id: %d", id);
```

The `%d` is a format specifier for an integer. Try this out and confirm that it fixes the problem.

16. Now compile and run the code with and without the `#pragma omp parallel for`. How many thread ids do you see in both cases?
17. The `#pragma omp parallel for` is separating the loop into different chunks so that each chunk can be done by a different processor simultaneously (in parallel). Which loop indexes are being handled by each thread? (Hint: print it out and see)
18. Sometimes we want the threads to each have their own copy of a variable and sometimes we want all the threads to share a copy. Are all the threads sharing the indexing variable for the loop, or do they each have their own copy in this case? To find out, print out the memory location where the indexing variable is stored. The format specifier for a memory address/pointer is `%p`, use this to see what the address is for the indexing variable inside the loop. You should find that each thread has its own memory address for the indexing variable if you have written the loop using something like `for (long long ind = 0; ind < SIZE; ind++)`. What do you think would happen if each thread/process had to share the same memory location for the indexing variable? (HINT: It is the same problem as what happened in #15 when all the threads were competing for a shared resource, printing to the same terminal window. When threads/processes are competing for a resource and it causes unpredictable outcomes depending on which thread or process gets there first, this is called a **race condition** and is generally considered undesirable!)

19. Given the number of processors on your machine, how much faster do you think this task should be if we use all of the processors?

Figure 7.5: CASPER CS1 Part 2 of Plugged-In OpenMP Array Creation Assignment

Part 3: Explore the `chrono` library and measure if the parallelization is producing any speedup in the code's runtime.

20. [AI] To actually measure how fast the code runs we need to take a look at another C++ library `chrono`. Use AI to find out about how to use this library to measure the execution time of a piece of code. The `<chrono>` library has several different types of clocks, so make sure you have a good understanding of what the different clocks are. Justify your choice of clocks. (Maybe try an AI prompt like "Help me learn to use `<chrono>` to benchmark my code".)
21. Figure out how to time the block of code that parallelizes the loop. Run the program several times using the `#pragma omp parallel for` and without using it. Are the times consistent for the same conditions? If not, why? The parallel for loop should be faster than the non-parallel version. Does it come close to the speed up that you predicted above in question 19? If not, why not? (You can use AI to help answer this question.)
22. How much time did this assignment take? Any other comments on how to make this assignment better for future students?

TURN IN: Your answers to the questions above and the code that you wrote and also how long this assignment took and any feedback that will make it better for future students.

Figure 7.6: CASPER CS1 Part 3 of Plugged-In OpenMP Array Creation Assignment

Prerequisites Students should have already worked with arrays in C++ and be able to write a loop that sums the contents of an array. The Plugged-In OpenMP Array Creation Assignment (shown in Section 7.4.1) is the prerequisite; The Unplugged Coin Addition Activity (shown in Section 7.3.3) was designed as a co-requisite.

Learning Outcomes

- Students will learn to recognize a race condition.
- Students will learn to recognize a shared variable.
- Students will describe why a simple loop parallelization using `#pragma omp parallel for` may fail

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests) Repository** with assignment and code: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper/Plugged_Activities/Plugged-In_OpenMP_Array_Sum
- **How-To Guides** Part 1 of the assignment, shown in Figure 17.7, asks students to write and then parallelize code using `#pragma omp parallel for` to sum the contents of an array. Since at this point, they only know how to parallelize a `for` loop that does not take into account shared variables (as in Section 7.4.1), they will try it this way and discover that this does not work, leading to questions about why this isn't working as desired. .

Introduction to parallel computing and OpenMP in C++ Part 2: shared variables

This activity is a pair programming exercise. This means that one person is at the keyboard (the driver) and the other is the navigator, observing, reviewing, and offering advice. This activity introduces the concept of a race condition and shared variables.

1. Starting with the code from [part 1 \(populating the array\)](#), that you did last week, add a new `for` loop that traverses the array, adding up all the values in the array using an accumulator. Check to make sure you are getting the correct answer using the formula for the sum of numbers from 1 to N is given by $N(N+1)/2$ (although the way we have populated the array, we are actually doing the sum from $0 + 1 + 2 + \dots + N-1$. Be sure that your code works for an N of 20,000,000. (This should be a non-parallel program with no openMP.)
2. Add the `#pragma omp parallel for` from Part1 to both loops in the program. Are you still getting the correct answer?
3. Change your array size N to a smaller value. Examine the accumulator variable inside the accumulation loop along with the thread id for each part of this loop. What is going on here?

TURN IN: Code that does this task in parallel and in serial. The serial code should work. Make sure you have implemented the formula to prove that it works and you get the correct value! Each block of code should also have timing information.

Figure 7.7: CASPER CS1 Part 1 of Plugged-In OpenMP Array Sum Assignment

In Part 2, students participate in the Unplugged Coin Addition Activity (see Section 7.3.3 which will illustrate that the naive approach to summing the array with a shared variable won't work in parallel). Ideally students will realize that this code can be parallelized if each of the threads are responsible for finding a partial sum of part of the array and the accumulator that stores the final result must be protected and only accessed by one thread at a time.

Part 3 of this activity is a demonstration explaining how to implement this idea using OpenMP. The key part of the code is shown in Figure 17.8

A video explaining the solution is at: <https://casper.yuja.com/V/Video?v=15996706&a=206700495>

Figure 7.8: CASPER CS1 Part 3 of Plugged-In OpenMP Array Sum Assignment

```
#pragma omp parallel //denotes the start of a parallel region
{
    long long int partialSum = 0; //each thread will have it's own copy

    #pragma omp for //use this inside a parallel region instead of "pragma omp parallel for"
    for (long long int j = 0; j < N; j++) {
        partialSum += array[j]; //each thread computes a local sum
    }

    #pragma omp critical //marks code that only one thread at a time can use
    {
        sum += partialSum;
    }
}
```

This activity is designed with multiple parts including student pair programming exploration, an unplugged group activity (Unplugged Coin Addition Activity, Section 7.3.3), and an instructor led demonstration.

Experience and Teaching Tips

This activity could be split up over several class sessions (completing one of the sections each class, for example), or it could be done in one class period, especially if the class is at least 90 minutes. The unplugged group activity will be difficult for on-line students to replicate, so on-line students may need to watch a video of the live activity or if possible, they could likely participate synchronously but remotely via video conference.

Data and Analysis

We will be working to improve this activity since it is not clear that students were achieving the desired outcomes. In the class in which this intervention was implemented, students wrote the code to parallelize the array summation (that should not have worked), but didn't really seem to be intrigued by why it wasn't working. It is possible that the sophistication level needed to recognize that this code is not working and be bothered by this outcome is not found as readily at the CS1 level. This activity was more effective at the CS2 level, which we tried in Spring 2026 (see Section 17.4.2). It is also possible that the activity directions could be improved to guide students to discovering that the code does not work properly. Students should also be prompted to brainstorm reasons for why the code does not work.

7.4.3 Plugged-In Distributed Data USGS API Earthquake Assignment

Problem Description, Prerequisites, and Learning Outcomes

Problem Description This assignment was adapted from TNTECH's Plugged-In Earthquake Tracker Assignment (see Section 2.4.1).

Prerequisites Students should be familiar with functions. This assignment assumes knowledge of cURL, JSON, and C++ functions. Students in this class were given mini-assignments on cURL and JSON before this assignment. (These are in the github repository for this assignment.)

Learning Outcomes

- **Define distributed data concepts** Students will explore how modern applications often rely on databases located on remote networks, requiring programs to interact with remote services to obtain information.
- **API Interaction** Students will learn what an API is and how to use it to request specific data.
- **Introduction to cURL and libcurl** Students will learn to use cURL as both a command line tool and a C++ library (libcurl) for transferring data from internet servers.
- **JSON structure** Students will learn about JSON formatting in key/value pairs and use the json.hpp library to parse and manipulate the data within a C++ environment.

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/Chapter7-Casper/Plugged_Activities/Plugged-In_Distributed_Data
- **How-To Guides** In this assignment the students are given some sample code that shows how to use the NumbersAPI <https://publicapi.dev/numbers-api> to retrieve trivia about a number using the `curl.h` library along with a video explanation of the code. Next, students are given some sample code that uses the `json.hpp` library to create a JSON object that can be indexed with by the fields returned from the JSON request in the cURL. The end goal is for students to be able to use the sample code to interact with the USGS API to create a program that will allow a user to retrieve earthquake data for earthquakes above a user entered magnitude. Since this was developed for an on-line class, it was designed to be a step-by-step activity that students could work through on their own, but this could also be used in a live class as a flipped classroom or paired programming lab assignment.

Experience and Teaching Tips

A major potential problem with any distributed data assignment is if the API changes or becomes unavailable the assignment won't work anymore. For example, as of the writing of this section, the NumbersAPI used the example code was unavailable, so these materials may or may not work depending on whether or not the NumbersAPI is available. The USGS API appears more stable, but is more complex, so it is desirable to start with an easier API.

Data and Analysis

Since these materials were used in an on-line class with only 2 students, there was limited and anecdotal data to report. Both students in the class completed the assignment satisfactorily and found it to be a challenging assignment. One student expressed enthusiasm about the real-world application.

In future iterations of this assignment, the language will be changed to Python and a more stable API (other than NumbersAPI) will be found. Python is has more accessible packages for distributed data that would not require discussion of cURL. JSON would still need to be taught, but flows more smoothly from Python dictionaries. Exposure to Python would be a prerequisite, but at Casper College most students have been exposed to Python in CS0, so this would not be a barrier. The added benefit for students would be for them to be given more exposure to Python code. This assignment might be a better match for a CS2 class, however, we were not able to figure out an appropriate time in the semester to put this content. A better fit for this content might actually be an introduction to Web App Development. At Casper College, the Web App Development course has CS1 as a prerequisite, so students would have more sophistication, and the content seems to fit within the content Web App better than the content of CS1 or CS2. This solution would keep the overall goal of exposing students to concepts in PDC in the first two years of undergraduate computer science.

7.4.4 Plugged-In Physical Computing 1 Introduction and Traffic Light

Problem Description, Prerequisites, and Learning Outcomes

Description. This activity does not directly involve any PDC concepts, but it is required to complete the rest of the physical computing projects that do have PDC content. This project introduces students to using the GPIO pins to control a blinking light. Students are introduced to the `gpiozero` Python package (<https://gpiozero.readthedocs.io/en/stable/#>) that allows simplified access to the GPIO pins. Students are walked through the process of setting up a basic LED circuit on a breadboard and connecting it to the correct GPIO pin on the Raspberry Pi. The hardware set up is shown in Figure 7.9. Students are given some starter code that utilizes the `gpiozero` Python package to control the rate at which the LED in the circuit blinks. Students are then asked to take the project one step further by constructing a circuit and software that can control 3 LEDs to create a traffic light.

Prerequisites prior experience with Python.

Learning Outcomes

- Students will learn how to identify and use the 40 pins on a Raspberry Pi.
- Students will learn about the pin types on a Raspberry Pi.
- Students will practice setting up circuits using breadboard, jumper wires, resistors, and LEDs.

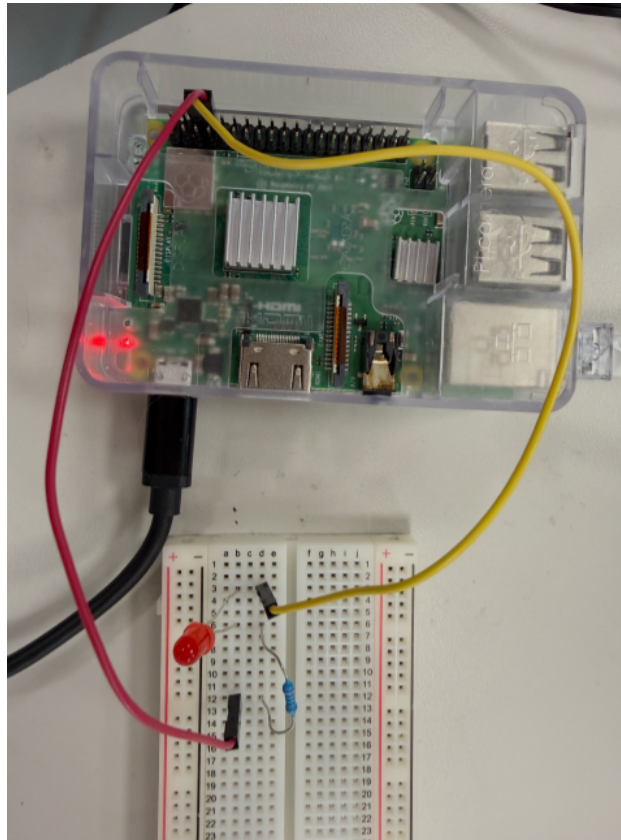


Figure 7.9: CASPER CS1 Hardware Circuit for Physical Computing Project 1

- Students will learn how to use the `gpiozero` Python package to control hardware elements such as LEDs.

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** In addition to a Raspberry Pi, students will need a breadboard, 3 resistors, 3 LEDs of different colors, and jumper wires. We created kits that had all of these components for students to borrow. The complete directions are found at: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper/Plugged_Activities/Physical_Computing_1
- **How-To Guides** Students who are unfamiliar with breadboards and electronic components will need explanation to be successful at this activity.

Experience and Teaching Tips

A concern with this activity is that the on-line students needed to overcome additional hurdles in acquiring the hardware kit and becoming familiar with the components, since they might not be at all familiar with circuitry.

Data and Analysis

This activity seemed to go pretty smoothly with most in-class students finishing in 20–30 minutes, mostly without instructor assistance. Only one of the three on-line students completed the assignment.

7.4.5 Plugged-In Physical Computing 2 Buttons and Multiple Processes

Problem Description, Prerequisites, and Learning Outcomes

Description. This project adds a button and code that monitors the button (“when pressed”) and (“when released”) so that when the button is pressed the LED is turned on and when the button is released the LED is turned off. The project guides students through changing the code so that when the button is pressed, the LED will continually flash a message in Morse code. All of the above tasks are possible without parallel processes. But if we want to change the program so that when we hit the button the message starts sending and when we hit the button again it stops, this requires multiple processes: one to send the message, one to monitor the button. The students are guided to use the Python multiprocessing package to manage these two processes. The code shown in Figure 7.10 is given in the assignment as an example of using the multiprocessing package in Python to manage multiple processes. The students are asked to take this work one step further by adding another button so that the resulting project will have button 1 result in message 1 from the LED and button 2 will result in a different message being transmitted by the LED.

Prerequisites. Physical Computing 1 (Section 7.4.4).

Learning Outcomes

- Students will use event-driven programming using a physical button
- Students will use the multiprocessing package to launch, monitor, terminate and join separate processes

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/Chapter7-Casper/Plugged_Activities/Physical_Computing_2
- **How-To Guides** This assignment is a multi-step walk through that guides the student toward building the starter code and circuit from scratch, and then asks the student to add on a new feature to an existing circuit. This assignment would also be great to do as a paired programming exercise, especially if more hardware-oriented students (electrical and computer engineering students, for example), could be paired with computer science students.

```

import multiprocessing
from time import sleep
from gpiozero import LED, Button
from signal import pause

red = LED(14)
button = Button(17)
process = None

def message():
    # code to transmit the Morse Code message
    while True:
        red.on()
        sleep(1)
        red.off()
        sleep(1)

def button_callback():
    global process
    print("Button pressed")

    if process and process.is_alive():
        print("stopping message")
        process.terminate()
        process.join()
        red.off()
    else:
        print("launching message")
        process = multiprocessing.Process(target=message)
        process.start()

button.when_pressed = button_callback

print("Ready to run")
pause()

```

Figure 7.10: CASPER CS1 Physical Computing project 2 given code for instruction on the python multiprocessing package

Experience and Teaching Tips

This assignment is definitely more difficult than Physical Computing 1 and required more time for students to complete. Since this activity builds on the skills introduced in Physical Computing 1, it is suggested that there should not be too much time between these two activities.

Data and Analysis

In the spring semester 2025, all four students in the in-person class completed this assignment and one out of three on-line student finished the assignment (the same students who completed Physical Computing 1).

7.4.6 Plugged-In Physical Computing 3 Temperature Sensor

Problem Description, Prerequisites, and Learning Outcomes

Description In this project, we add a *DHT22* temperature and humidity sensor to the circuit.

Prerequisites Physical Computing 1 and 2 **Learning Outcomes**

- Students will learn to use a virtual environment (venv)
- Students will learn how to use the `adafruit-circuitpython-dht` library to read data from the DHT22 temperature and humidity sensor
- Students will manage multiple processes running simultaneously with the multiprocessing package
- Students will learn how to use a shared object so that separate processes can have access to the same object
- Students will be exposed to error handling using try-except blocks

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** Materials for this activity are found at: https://github.com/CDER-Center/CS1-CS2_Exampleplar_Ebook/tree/main/CS1/chapter7-Casper/Plugged_Activities/Physical_Computing_3
- **How-To Guides** Part 1 of the assignment walks students through setting up the circuit to use the sensor. Students set up a virtual environment to install the needed libraries to control the sensor. None of the previous classwork that students have done at Casper College would have required a virtual environment, so this definitely needed some explanation. We also provided the test code to test the sensors. It turns out that the sensors have a high rate of not working, so it was important to find out early if the sensors are even working properly. There is also some sophisticated exception handling that is necessary for this sensor, so all of that code had to be developed, provided to students, and explained to the students in the materials. After all of this preliminary work is done, students should be able to develop a circuit and software to solve the following scenario:

You are designing a system that can monitor the temperature in either a greenhouse or a data center. The system should allow a temperature threshold to be toggled by pressing a button. If the temperature threshold is set for the greenhouse, pressing the

button should set the threshold for the data center. Presumably the threshold for the greenhouse is higher than the temperature for the data center. If the temperature is higher than whatever the current threshold is set to, the LED should blink out a custom pattern.

The resulting solution needs to have:

- A process that is monitoring the *DHT22* sensor for the temperature value.
- A process that is monitoring the button that will toggle the threshold value that triggers the led blinking function.
- A process that is running the led blinking function if the temperature is above the threshold value.
- The solution also needs to have a shared object that is storing a shared value (the threshold) since when the multiprocessing package is used, each process gets its own copy of the entire script using the global variables.

Because this code turned out to be a lot more complex than the first two projects, students are given starter code to complete instead of starting from scratch.

Experience and Teaching Tips

This was definitely the longest of the Physical Computing assignments and probably should be broken up into multiple class periods and/or a longer time frame to completion. One major problem with this project is the use of the DHT22 sensor. It is not completely reliable. One of our 5 sensors had to be replaced because it just wasn't working at all. The sensor also doesn't have very fast response time. Holding on to the sensor with your hand will result in the temperature being raised, but even after you release the sensor, the temperature does not go down very fast, so testing programs is quite laborious. The Python multiprocessing package also has a steep learning curve for both myself and the students.

Data and Analysis

All four students who ultimately passed the class completed this assignment successfully. Because of the steep learning curve for the parts of this assignment, it might be better to break the assignment up into more parts.

7.5 Course-wide Pre and Post Course Surveys

Course-wide pre and post surveys were given to students in the Spring 2025 (No PDC interventions) and Spring 2026 (PDC interventions as described) semesters. The common survey questions used in this project are described in Section 1.5). As mentioned previously, all three classes discussed in this section were small. The Spring 2025 semester had 13 students with 3 on-line and 10 in the live class. Of this population, 5 students completed both the pre-course survey and the post-course survey and also gave consent for data to be used for this study. The Spring 2026 semester had 7 students with 3 on-line and 4 in the

live class. In this class, 3 students completed both pre- and post-course surveys and gave consent. The survey was not given to the Summer 2025 class since there were only 4 students who started the class and that number quickly dwindled to 2 students. The results of the surveys are summarized in Tables 7.4, 7.5, and 7.8.

No noticeable decrease in perception of learning in non-pdc topics. An important question that should be asked is whether inclusion of PDC topics has resulted in a decrease in learning gains for the core CS1 topics. Figure 7.4 summarizes the survey results for questions that asked students about their experience using various programming languages and programming skills and concepts in their programs. A sample question is shown in Figure ??.

Table 7.4: CASPER CS1 Survey Results for Experience Questions

Experience	No intervention			Intervention		
	R_{base_pre}	R_{base_post}	$Mdif_base$	R_{int_pre}	R_{int_post}	$Mdif_int$
Computer experience	5.60	4.40	-1.20	5.33	5.00	-0.33
C	1.00	1.00	0.00	1.33	3.33	2.00
C++	1.20	3.40	2.20	1.50	4.00	2.50
C#	1.00	1.00	0.00	1.33	3.00	1.67
Java	1.00	1.00	0.00	2.00	1.33	-0.67
Javascript	1.00	1.20	0.20	1.67	1.33	-0.33
PHP	1.00	1.00	0.00	1.67	1.33	-0.33
Python	2.80	3.80	1.00	4.33	5.00	0.67
Scratch	1.40	1.80	0.40	4.00	3.00	-1.00
Visual Basic	1.20	1.40	0.20	1.67	1.00	-0.67
other	1.40	1.00	-0.40	2.00	2.00	0.00
data types	2.20	4.40	2.20	4.00	5.67	1.67
arithmetic expressions	2.20	4.00	1.80	4.00	5.67	1.67
branching	2.60	4.20	1.60	4.67	5.67	1.00
loops	3.00	4.20	1.20	4.67	5.33	0.67
functions	3.00	4.20	1.20	4.33	5.33	1.00
reference variables	2.80	3.40	0.60	4.67	4.67	0.00
arrays	1.60	4.00	2.40	3.00	4.67	1.67
pointers	1.40	3.00	1.60	1.67	4.33	2.67
C++ structures	1.40	3.20	1.80	1.00	5.00	4.00
classes & objects	1.40	3.40	2.00	2.00	4.33	2.33

The CS1 topics that were covered in both classes are shown highlighted in gray. The raw responses (on a 7-point Likert scale) for the pre survey R_{base_pre} and post survey R_{base_post} during the baseline Spring 2025 semester are shown in the first two columns for questions in which students were asked to rate their experience with programming languages and the use of various programming topics in their programs. The mean of the difference of the post-pre matched survey responses are in the columns labeled $Mdif_base$ column. The number of responses for the baseline year with pre and post matches was 5. The last three columns show the responses and the mean difference for the Spring 2026 semester in which

...

Please rate your experience with **using pointers (including dynamic memory allocation)** in your programs:

1 2 3 4 5 6 7

Not experienced at all ☐ ☐ ☐ ☐ ☐ ☐ ☐ Extremely experienced

Figure 7.11: CASPER CS1 Sample Survey Question Reflecting on Level of Experience

the PDC interventions described in this chapter were implemented. The number of responses with pre and post matching for the intervention semester was 3. The rows highlighted in gray are topics that were presented in the course. Both the No PDC Intervention class and the PDC Intervention class rated themselves as more experienced in all of these topics except “reference variables”, which showed the least gain in both classes and no gain in the PDC Intervention class, which may be due to the fact that this group also rated their pre experience higher than the No Intervention class. In fact, most of the topics covered in the class had a perceived gain of at least 1.0. In contrast, very few of the topics/languages that were not covered in the class showed gains, which is to be expected. The exception to this statement are:

- 1.67 gain reported in C# by the PDC Intervention class. Other researchers in this group have reported similar reported gains by classes. Our hypothesis is that students are assuming that learning about C++ will translate into knowledge of C#. This group also reported higher levels of experience with C# in general, so it is also possible that this class gained some experience in C# through some other means during the course of the semester.
- 1.0 loss reported by the PDC Intervention class in the Scratch. This effect was due to one of the three students reducing their scores from 4 to 2, the other two students gave the same experience score for both pre and post.

The summary of survey results that asked students about experience with PDC topics is shown in Table 7.5. In this table, raw responses (on a 7-point Likert scale) for the pre survey R_{base_pre} and post survey R_{base_post} during the baseline Spring 2025 semester are shown in the first two columns for questions in which students were asked to rate their experience with programming languages and the use of various programming topics in their programs. The mean of the difference of the post-pre matched survey responses are in the columns labeled $Mdif_base$ column. The number of responses for the baseline year with pre and post matches was 5. The last three columns show the responses and the mean difference for the Spring 2026 semester in which the PDC interventions described in this chapter were implemented. The number of responses with pre and post matching for the intervention semester was 3. Encouragingly, the PDC Intervention class showed large increases in experience level for each of the PDC topics covered. The No PDC Intervention

class also indicated gains in these topics (even without the class interventions), but the magnitudes were smaller.

Table 7.5: CASPER CS1 Survey Results Experience with PDC Topics

Experience	No intervention			Intervention		
	R_{base_pre}	R_{base_post}	$Mdif_base$	R_{int_pre}	R_{int_post}	$Mdif_int$
parallel processing	1.00	2.00	1.00	2.00	5.00	3.00
distributed computing	1.00	1.20	0.20	1.33	5.33	4.00
event handling	1.00	1.80	0.80	3.00	5.00	2.00

Students were also asked to rate their level of interest in computing and in PDC. These survey questions are shown in Figure ?? and a summary of the results are shown in Table 7.6.

Figure 7.12: CASPER CS1 Sample Survey Question Reflecting on Level of Interest in Computing and PDC

Table 7.6: Survey Results for CS1 Interest Questions- Casper College

Interest	No intervention			Intervention		
	R_{base_pre}	R_{base_post}	$Mdif_base$	R_{int_pre}	R_{int_post}	$Mdif_int$
computing	5.40	5.20	-0.20	6.33	6.33	0.00
PDC	4.20	4.20	0.00	5.67	6.00	0.33

For Table 7.6, the raw responses (on a 7-point Likert scale) for the pre survey R_{base_pre} and post survey R_{base_post} during the baseline Spring 2025 semester are shown in the first two columns for questions in which students were asked to rate their Interest in computing and interest in PDC topics. The mean of the difference of the post-pre matched survey responses are in the columns labeled $Mdif_base$ column. The number of responses for the baseline

year with pre and post matches was 5. The last three columns show the responses and the mean difference for the Spring 2026 semester in which the PDC interventions described in this chapter were implemented. The number of responses with pre and post matching for the intervention semester was 3. These results show a generally high level of interest in both computing and PDC in both the pre and post surveys. A similar observation can be made about the results of asking students about their level of agreement with the statement "In general, I believe learning about parallel and distributed computing is important" as shown in Table 7.7.

Table 7.7: CASPER CS1 Survey Results Interest Questions

	No intervention			Intervention		
Importance	R_{base_pre}	R_{base_post}	$Mdif_base$	R_{int_pre}	R_{int_post}	$Mdif_int$
importance - PDC	4.00	4.75	0.75	6.00	6.00	0.00

For Table 7.7, the raw responses (on a 7-point Likert scale) for the pre survey R_{base_pre} and post survey R_{base_post} during the baseline Spring 2025 semester are shown in the first two columns for questions in which students were asked the level of agreement with the following statement: "In general, I believe learning about parallel and distributed computing is important." The mean of the difference of the post-pre matched survey responses are in the columns labeled $Mdif_base$ column. The number of responses for the baseline year with pre and post matches was 5. The last three columns show the responses and the mean difference for the Spring 2026 semester in which the PDC interventions described in this chapter were implemented. The number of responses with pre and post matching for the intervention semester was 3.

Table 7.8: CASPER CS1 Survey Results for Understanding Questions

	No intervention			Intervention		
Understanding	R_{base_pre}	R_{base_post}	$Mdif_base$	R_{int_pre}	R_{int_post}	$Mdif_int$
computing	3.60	5.00	1.40	4.67	6.00	1.33
parallel and dis- tributed computing	2.40	3.40	1.00	3.33	5.67	2.33

For Table 7.8, the raw responses (on a 7-point Likert scale) for the pre survey R_{base_pre} and post survey R_{base_post} during the baseline Spring 2025 semester are shown in the first two columns for questions in which students were asked their level of agreement with the following statements: "I believe I have a good understanding of computing" and "I believe I have a good understanding of parallel and distributed computing." The mean of the difference of the post-pre matched survey responses are in the columns labeled $Mdif_base$ column. The number of responses for the baseline year with pre and post matches was 5. The last three columns show the responses and the mean difference for the Spring 2026 semester in which the PDC interventions described in this chapter were implemented. The number of responses with pre and post matching for the intervention semester was 3.

7.6 Conclusion

The experience of implementing PDC concepts in this chapter highlights a distinct trade-off between conceptual engagement and technical implementation hurdles. The unplugged activities, such as the Penny Search and Coin Addition were highly engaging to students. These exercises were also not difficult to prepare or implement and also did not require a lot of class time. They were successful at building an intuitive foundation for fundamental PDC concepts and vocabulary. The difficulty in this material is transitioning to higher impact assignments that reach higher Bloom's levels such as "Analyze" or "Evaluate". The OpenMP based activities had a steep learning curve that forced the instructor to provide most of not all of the code. This was also the case for the USGS Distributed Data assignment that was not used in the Spring 2026 semester because it was just too long and did not seem to fit with the Gaddis-based materials as well as the zyBooks materials used in summer 2025. The physical computing assignments were particularly promising because they required students to break away from sequential programming by managing multiple processes and event-driven tasks. Unfortunately, these assignments also proved to have a steep learning curve. The frustration of this experience was reflected in a blunt post-course survey response where one student labeled these assignments as "the one dud" of the year:

While conceptually it would've been great for teaching parallelization, the unreliability of the raspberry pi devices, combined with using older text editors for a 'newer' language like python left me feeling like I was fighting the raspberry pi and learning more about electrical engineering than learning parallelization"

It is clear that more work will need to be done to better scaffold the plugged-in assignments to reduce the technical overload while keeping the goal of reaching higher levels of Bloom's taxonomy.

Bibliography

- [1] Tony Gaddis. Starting Out with C++: From Control Structures through Objects. Pearson, 10th edition, 2021. Revel Edition.
- [2] zyBooks. Programming in C++ with zyLabs. <https://www.zybooks.com>. Interactive online textbook.

Appendix

|You must include these four specific subsections (in addition to anything else):|

Github Link to Instructional Material

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper

Detailed Pre and Post Syllabus

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper/Detailed_Pre_and_Post_Syllabus

Organization of Material

Materials in the Github repository are organized as follows;

- Detailed_Pre_and_Post_Syllabus – This directory contains information on the baseline semester of the course before the PDC interventions were added (Spring 2025) and the post semester when the PDC interventions were added (Spring 2026).
- Evaluation_Data_and_Analysis – This directory has documentation concerning the instruments and data collected during the course of this effort.
- Plugged_Activities – This directory contains resources used to implement the PDC Activities that involved coding activities.
 - Physical_Computing_1 – This directory contains the resources used to implement Physical Computing 1 Introduction and Traffic Light as described in Section 7.4.4.
 - Physical_Computing_2 – This directory contains the resources used to implement Physical Computing 2 Buttons and Multiple Processes as described in Section 7.4.5.

Survey and Other Anonymized Data and Analysis

https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter7-Casper/Evaluation_Data_and_Analysis